Software Requirements Specification

for

BUP Program Outcome Mapping System

Version 1.0 approved

Prepared by

Jawad Anzum Fahim(2254901022)

Ruponti Muin Nova(2254901038)

Date:07/07/2025

Table of Contents

Table of Contents ii
Revision History iii

1. Introduction 4
1.1. Purpose 4
1.2. Document Conventions 4
1.3. Intended Audience and Reading Suggestions 4
1.4. Project Scope 5
1.5. References 5

2. Overall Description 6
2.1. Product Perspective 6
2.2. Product Features 6
2.3. User Classes and Characteristics 6
2.4. Operating Environment 7
2.5. Design and Implementation Constraints 7
2.6. User Documentation 8
2.7. Assumptions and Dependencies 8

3. External Interface Requirements 9
3.1. User Interfaces 9
3.2. Hardware Interfaces 9
3.3. Software Interfaces 10
3.4. Communications Interfaces 10

4. System Features 10
4.1. User Authentication 10
4.1.1. Description and Priority 10
4.1.2. Stimulus/Response Sequences 10
4.1.3. Functional Requirements 11
4.2. Teacher Functions 11
4.2.1. Description and Priority 11
4.2.2. Stimulus/Response Sequences 11
4.2.3. Functional Requirements 11
4.3. Administrator Functions 12
4.3.1. Description and Priority 12
4.3.2. Stimulus/Response Sequences 12
4.3.3. Functional Requirements 12

5. Other Nonfunctional Requirements 12
5.1. Performance Requirements 12
5.2. Safety Requirements 12
5.3. Security Requirements 13
5.4. Software Quality Attributes 13
5.5. Business Rules 13

6. Other Requirements 13
6.1. Database Requirements 13
6.2. Internationalization Requirements 13
6.3. Legal and Compliance Requirements 14

Appendix A: Glossary 15

Revision History

Name	**Date**	**Reason For Changes**	**Version**

1. Introduction

1.1 Purpose

This Software Requirements Specification (SRS) document describes the software requirements for the **Program Outcome Mapper**, release **version 0.1.0**. The scope of this document covers the entire system, including all features, functionalities, and modules. This encompasses the user-facing application for teachers, the administration portal for system administrators, and the backend API that powers both interfaces.

1.2 Document Conventions

This document adheres to the following typographical and formatting conventions:

➢ **Alignment:** The entire document should be justified.
➢ **Chapter Title:**
- **Font Face:** Times New Roman
- **Font Style:** Bold
- **Font Size:** 16

➢ **Sub-title:**
- **Font Face:** Times New Roman
- **Font Style:** Bold
- **Font Size:** 14

➢ **Running Text:**
- **Font Face:** Times New Roman
- **Font Size:** 12

In addition to formatting, the following conventions apply to the content:
➢ **Requirement Identification:** All software requirements will be explicitly stated and uniquely identifiable.
➢ **Priorities:** Future additions, such as requirement priorities, will be clearly marked for each individual requirement statement.

1.3 Intended Audience and Reading Suggestions

This document is intended for a wide range of stakeholders, including:

➢ **Project Managers:** For understanding the project scope, objectives, and for project planning.
➢ **Developers:** As a guide for implementing the specified features and functionalities.
➢ **Testers and QA Engineers:** To create test plans, cases, and scripts to verify the software against its requirements.
➢ **Documentation Writers:** To create user manuals and help guides.

➢ **System Administrators:** To understand the system's architecture and maintenance requirements.
It is recommended to read this document in the following sequence:
1. **Section 1 (Introduction):** Provides an overview of the project and document.
2. **Section 2 (Overall Description):** Describes the product context, user characteristics, and constraints (to be created).
3. **Section 3 (Specific Requirements):** Details the functional, non-functional, and interface requirements. Readers can focus on the sections most relevant to their roles.

1.4 Product Scope

The **BUP Program Outcome Mapping System** is a full-stack MERN web application designed to provide educational institutions with a centralized platform for tracking and managing student academic performance against predefined course objectives(CO).

The primary purpose is to streamline the process of outcome-based education assessment. The key benefits and objectives include:

➢ **Centralized Data Management:** Consolidating all data related to courses, students, assessments, and outcomes in a single database.
➢ **Role-Based Access:** Providing distinct, secure interfaces for Teachers and Administrators to perform their specific duties.
➢ **Efficient Score Tracking:** Enabling teachers to easily input and save student scores for various assessment types.
➢ **Data-Driven Insights:** Allowing for the generation of score summaries to help evaluate student and program performance.
➢ **Streamlined Administration:** Empowering administrators to manage core system data such as teacher records and academic sessions.

This software supports the strategic goal of improving educational quality assurance by providing clear, auditable data on program outcomes.

1.5 References

The following documents and resources are referenced in this SRS or provide supplementary information:

1. **Project `README.md`:** Contains a high-level overview of the project, technology stack, and setup instructions. Located in the root directory of the project.
2. **Project `package.json`:** Lists all project dependencies and scripts. Located in the root directory of the project.
3. **Next.js Documentation:** (https://nextjs.org/docs) Visited Date: 02/05/2025
4. **MongoDB Documentation:** (https://www.mongodb.com/docs) Visited Date: 06/05/2025

2. Overall Description

2.1 Product Perspective

The **BUP Program Outcome Mapping System** is a new, self-contained web application. It is designed from the ground up to serve the specific need of tracking educational outcomes and does not replace a legacy system, nor is it a follow-on member of an existing product family.

The system is composed of three primary components that work together: a User/Teacher-facing web interface, an Administrator-facing web interface, and a centralized backend API that serves both. The entire system interfaces with an external MongoDB database for data persistence.

2.2 Product Functions

The major functions that the Program Outcome Mapper provides are summarized below.

For Teachers / General Users:

➢ User Authentication
- Login
- Logout
- Password Reset

➢ Management of Course Objectives

➢ Entry and submission of student assessment scores

➢ Viewing of consolidated score summaries

For Administrators:

➢ Administrator Authentication
- Login
- Logout

➢ Management of Teacher accounts
- Adding new teachers
- Viewing teacher details

➢ Management of academic sessions
- Creating and updating sessions

➢ Access to an administrative dashboard for system oversight

2.3 User Classes and Characteristics

The product is designed for two primary classes of users, distinguished by their privileges and system interactions.

Characteristic	Teacher / General User	Administrator
Primary Goal	To manage course data, input student scores, and	To manage the overall system, maintain user

	track academic progress for their courses.	records, and configure academic parameters.
Frequency of Use	Frequent, especially during active academic periods.	Intermittent, as needed for administrative tasks.
Product Functions Used	Score entry, score summary, course objective management.	Teacher management, session management, admin dashboard.
Technical Expertise	Basic computer and web browser proficiency. No specialized technical knowledge required.	Moderate technical proficiency. Understands system structure and data management principles.
Privilege Level	Standard user with access only to their own relevant data and courses.	Elevated privileges with the ability to create and modify system-wide data (teachers, sessions).

The **Teacher** is the most critical user class, as their regular input of data is the primary function of the application. The **Administrator** is crucial for ensuring the system is correctly configured and populated with the necessary foundational data.

2.4 Operating Environment

The software will operate in a standard client-server web environment.

➢ **Server-Side Environment:**

- **Operating System:** OS-agnostic; can be deployed on any system that supports Node.js (e.g., Linux, Windows Server, macOS).
- **Runtime Environment:** Node.js v20 or later.
- **Database:** Requires a network connection to a MongoDB v6.16.0 (or compatible) database instance.

➢ **Client-Side Environment:**

- **Platform:** Any desktop or mobile device with a modern web browser.
- **Operating System:** Any (e.g., Windows, macOS, Linux, iOS, Android).
- **Web Browser:** Latest stable versions of Google Chrome, Mozilla Firefox, Apple Safari, or Microsoft Edge.

2.5 Design and Implementation Constraints

The development and deployment of the Program Outcome Mapper are subject to the following constraints:

➢ **Technology Stack:** The application must be built using the specified technology stack: Next.js (v15.3.3), React (v19.0.0), TypeScript (v5), and Tailwind CSS (v4.1.10). This ensures consistency and leverages modern web development practices.

➢ **Database System:** The system is designed to work exclusively with MongoDB (v6.16.0 or compatible). All data models and queries must be compatible with MongoDB's document-based structure.
➢ **Authentication:** User authentication and session management must be implemented using JSON Web Tokens (JWTs) via the jose library. Passwords must be securely hashed using bcrypt.
➢ **Platform:** The server-side application must be deployable on a Node.js (v20 or later) runtime environment.
➢ **API Structure:** The backend must be implemented as a set of RESTful API endpoints within the Next.js pages/api/ directory structure.

2.6 User Documentation

The primary user documentation for the Program Outcome Mapper is the README.md file, which is delivered with the software. This document serves as a comprehensive guide for developers and system administrators.
The README.md file includes:
➢ **Project Overview:** A high-level description of the application's purpose.
➢ **Technology Stack:** A list of all major frameworks, libraries, and technologies used.
➢ **Project Structure:** An explanation of the codebase organization.
➢ **Core Features:** A detailed breakdown of functionalities available to both Teachers and Administrators.
➢ **API Endpoints:** A complete list and description of all available API routes.
➢ **Setup and Installation Guide:** Step-by-step instructions for setting up the development environment and running the project.
No other formal user documentation, such as user manuals or online tutorials for non-technical end-users, is scheduled for delivery with the current version. The user interfaces are designed to be intuitive and self-explanatory.

2.7 Assumptions and Dependencies

The requirements outlined in this SRS are based on the following assumptions and dependencies:
➢ **Assumptions:**
It is assumed that users (Teachers and Administrators) have basic computer and web browser proficiency.
The hosting environment (e.g., a server supporting Node.js) and the MongoDB database instance will be provided and maintained by the deploying institution.
Administrators are expected to provide accurate and valid data for system setup (e.g., teacher details, academic sessions).
The third-party email service used for sending password reset OTPs is assumed to be correctly configured and operational.
➢ **Dependencies:**
The project's stability is dependent on its open-source third-party components, including Next.js, React, and other libraries listed in the package.json file.
The system requires a persistent and stable network connection to the external MongoDB database.

The password reset feature is dependent on the reliability of the external Nodemailer service.

3. External Interface Requirements

This section details the various interfaces of the Program Outcome Mapper, including user, hardware, software, and communication interfaces.

3.1 User Interfaces

The application provides two distinct Graphical User Interfaces (GUIs), both accessed through a modern web browser:
➢ **Teacher/General User Interface:** This interface provides functionalities for managing course data, entering student scores, and viewing academic progress. Key pages include the user dashboard, assessment score entry form, and score summary view.
➢ **Administrator Interface:** This is a separate interface for system administrators to manage core system data, including teacher accounts and academic sessions.

GUI Standards:
➢ **Layout:** The interface is built with React and styled with Tailwind CSS, ensuring a responsive and consistent layout across the application.
➢ **Navigation:** A persistent navigation bar provides easy access to all primary features available to the logged-in user role.
➢ **Error Handling:** Error messages are displayed contextually within the interface to provide immediate feedback to the user.
➢ **Accessibility:** The UI is designed to be intuitive and usable for individuals with basic computer proficiency, requiring minimal training.
A detailed user interface specification document is not provided; the design prioritizes simplicity and established web conventions.

A detailed user interface specification document is not provided; the design prioritizes simplicity and established web conventions.

3.2 Hardware Interfaces

The Program Outcome Mapper is a standard web application and does not interface with any specialized hardware.
➢ **Client-Side:** The application is accessible on any hardware device capable of running a modern web browser, including desktops, laptops, tablets, and smartphones. No specific hardware configurations are necessary.
➢ **Server-Side:** The backend application runs in a Node.js environment and can be hosted on any standard server hardware that meets the OS and runtime requirements.

There is no direct data or control interaction between the software and specific hardware components.

3.3 Software Interfaces

The system interfaces with several software components to function correctly:
➢ **Database:** The application connects to a **MongoDB (v6.16.0 or compatible)** database instance for all data persistence. This includes storing user credentials, course information, student scores, and session data. The connection logic is managed by the application backend.
➢ **Third-Party Libraries:** The project relies on several key open-source libraries, as defined in the package.json file:
- Next.js & React: The core framework for the frontend and backend.
- jose: Used for creating and verifying JSON Web Tokens (JWTs) for secure session management.
- Nodemailer: Used as a software interface to an email sending service for dispatching password reset OTPs.
➢ **Operating System:** The server is OS-agnostic and can be deployed on any system supporting Node.js (e.g., Linux, Windows Server).

3.4 Communications Interfaces

➢ **Web Browser Communication:** The primary communication interface with the user is via a web browser. The application is designed to communicate over **HTTPS**, ensuring that all data exchanged between the client and server is encrypted.
➢ **API Protocol:** The frontend client communicates with the backend server by making calls to a **RESTful API**.
Protocol: HTTP/S
Data Format: All data payloads are sent and received in **JSON** format.
Authentication: API routes are secured using JWTs, which are passed in browser cookies with each request.
➢ **Email Communication:** For the password reset feature, the system communicates with an external SMTP server via the Nodemailer library. It sends an email containing a secure OTP to the user's registered email address. Message formatting is simple HTML/text.

4. System Features

This section describes the functional requirements of the system, organized by major features.

4.1 User Authentication

4.1.1 Description and Priority

This feature encompasses the verification of user identity for both Teachers and Administrators, providing secure, role-based access to the system, and handling account recovery. Priority: High

4.1.2 Stimulus/Response Sequences

➢ **User Login:** A user (Teacher or Admin) enters their email and password into the corresponding login form and submits. The system validates the credentials. On success, the system logs the user

in, creates a session token, and redirects them to their dashboard. On failure, the system displays an error message.

➢ **Password Reset Request:** A user enters their email address into the forgot password form. The system validates the email, generates a One-Time Password (OTP), and sends it to the user's email address.

➢ **Password Reset Execution:** The user provides the OTP and a new password. The system validates the OTP and, if valid, updates the user's password in the database.

4.1.3 Functional Requirements

➢ **REQ-1:** The system shall provide a secure login interface for Teachers/General Users at the /login route.

➢ **REQ-2:** The system shall provide a separate, secure login interface for Administrators at the /admin/login route.

➢ **REQ-3:** The system shall use JSON Web Tokens (JWTs) stored in browser cookies to manage user and admin sessions.

➢ **REQ-4:** The system shall provide a password reset function that sends a time-sensitive OTP to a user's registered email address.

➢ **REQ-5:** The system must enforce access control by restricting access to teacher-specific routes and admin-specific routes based on the user's role.

4.2 Teacher Functions

4.2.1. Description and Priority

This feature set allows teachers to manage course-related data, primarily focusing on tracking student performance against course objectives. **Priority: High**

4.2.2. Stimulus/Response Sequences

➢ **Score Entry**: The teacher navigates to the assessment score page. The system displays a form with fields for session, course, student, and various assessment criteria. The teacher fills in the scores and submits the form. The system validates the data and saves it to the database.

➢ **View Score Summary**: The teacher navigates to the score summary page. The system retrieves and displays a consolidated summary of student scores.

4.2.3. Functional Requirements

➢ **REQ-7**: The system shall display a central dashboard at /homepage for authenticated teachers, providing access to core functionalities.

➢ **REQ-8**: The system shall allow teachers to view course objectives.

➢ **REQ-9**: The system shall provide an interface (/assessment_score) for teachers to input and save student scores for multiple assessment types.

➢ **REQ-10**: The system shall provide an interface (/score_summary) for teachers to view a consolidated summary of student scores.

4.3. Administrator Functions

4.3.1. Description and Priority

This feature set provides administrators with the tools to manage the overall system configuration, including user accounts and academic parameters. **Priority: High**

4.3.2. Stimulus/Response Sequences

➢ **Add Teacher**: The administrator navigates to the "Add Teacher" function from their dashboard. The system presents a form. The administrator enters the new teacher's details and submits. The system validates the input and creates a new teacher record in the database.

➢ **View Teachers**: The administrator navigates to the teacher details page. The system retrieves and displays a list of all teacher records.

➢ **Manage Sessions**: The administrator uses the session management interface to create or update an academic session. The system saves these changes to the database.

4.3.3. Functional Requirements

➢ **REQ-11**: The system shall provide a dedicated dashboard for administrators at /admin/homepage with links to management functions.

➢ **REQ-12**: The system shall allow administrators to add new teacher accounts to the system.

➢ **REQ-13**: The system shall allow administrators to view a complete list of all teacher details.

➢ **REQ-14**: The system shall allow administrators to create and manage academic sessions.

5. Other Nonfunctional Requirements

5.1 Performance Requirements

➢ **PR-1 (Page Load)**: All user-facing pages must render completely in the client's browser within 3 seconds on a standard broadband internet connection.

➢ **PR-2 (API Response)**: API endpoints should process requests and return a response within 2 seconds under normal load conditions.

➢ **Rationale**: These performance targets are intended to ensure a fluid and responsive user experience, minimizing wait times for users.

5.2 Safety Requirements

➢ **SA-1 (Data Integrity)**: The system must include safeguards to prevent accidental data loss or corruption, such as requiring user confirmation before permanent deletion of records.

➢ **SA-2 (No Physical Harm)**: As a web-based data management system, there are no direct safety requirements related to physical harm. The primary focus is on the safety and integrity of the data it stores.

5.3 Security Requirements

➢ **SE-1 (Authentication)**: All users must be authenticated via a unique username (email) and password before being granted access to any system functionalities.
➢ **SE-2 (Password Storage)**: All user passwords must be irreversibly hashed using a strong, one-way cryptographic algorithm (bcrypt) before being stored in the database.
➢ **SE-3 (Session Management)**: User sessions shall be managed securely using JSON Web Tokens (JWTs) that are validated on every request to a protected resource.
➢ **SE-4 (Access Control)**: The system must enforce strict role-based access control, preventing users from accessing any data or functionality outside of their designated role (e.g., Teachers cannot access Administrator pages or APIs).
➢ **SE-5 (Data Transmission)**: All communication between the user's web browser and the server must be encrypted using HTTPS to protect data in transit.

5.4 Software Quality Attributes

➢ **SQA-1 (Maintainability)**: The codebase shall be well-documented and adhere to the configured ESLint rules to ensure code quality, consistency, and ease of maintenance.
➢ **SQA-2 (Reliability)**: The system should be highly available, with a target uptime of 99.5%, especially during active academic periods.
➢ **SQA-3 (Usability)**: The user interface must be intuitive, responsive, and easy to navigate for users with basic computer skills, requiring minimal training.
➢ **SQA-4 (Portability)**: The server-side application is designed to be OS-agnostic, capable of running on any major operating system that supports Node.js.

5.5 Business Rules

➢ Only users with the "Administrator" role can create, view, or manage teacher accounts.
➢ Only users with the "Administrator" role can create or modify academic sessions.
➢ A user's role is fixed and determines their permissions throughout the application.
➢ A password reset OTP is valid for a limited, predefined period and can only be used once.
➢ Teachers can only enter and view scores for the courses and students assigned to them. (TBD: The mechanism for assigning teachers to courses is not yet defined).

6. Other Requirements

6.1. Database Requirements

➢ The application requires a connection to a MongoDB v6.16.0 (or compatible) database instance. The connection URI must be provided as an environment variable.

6.2. Internationalization Requirements

➢ The initial release of the Program Outcome Mapper will only support English (en-US). No provisions for internationalization or localization are included in the current scope.

6.3. Legal and Compliance Requirements

➢ There are no specific legal or compliance requirements (e.g., GDPR, FERPA) defined for the initial release. The system should be reviewed for compliance with local regulations if it is used to store sensitive student data.

Appendix A: Glossary

Term	Definition
API	Application Programming Interface: A set of rules and protocols for building and interacting with software applications.
GUI	Graphical User Interface: A visual interface that allows users to interact with electronic devices through graphical icons and visual indicators.
HTTPS	Hypertext Transfer Protocol Secure: An encrypted version of the HTTP protocol for secure communication over a computer network.
JWT	JSON Web Token: A compact, URL-safe means of representing claims to be transferred between two parties. Used for authentication and session management.
MongoDB	A NoSQL document-oriented database program used for high-volume data storage.
Next.js	A React framework for building server-rendered and statically generated web applications.
Nodemailer	A module for Node.js applications to allow for email sending.
OTP	One-Time Password: A password that is valid for only one login session or transaction.
RESTful	Representational State Transfer: An architectural style for designing networked applications, often used for creating web APIs.
SRS	Software Requirements Specification: A document that describes the what of a software system to be developed.

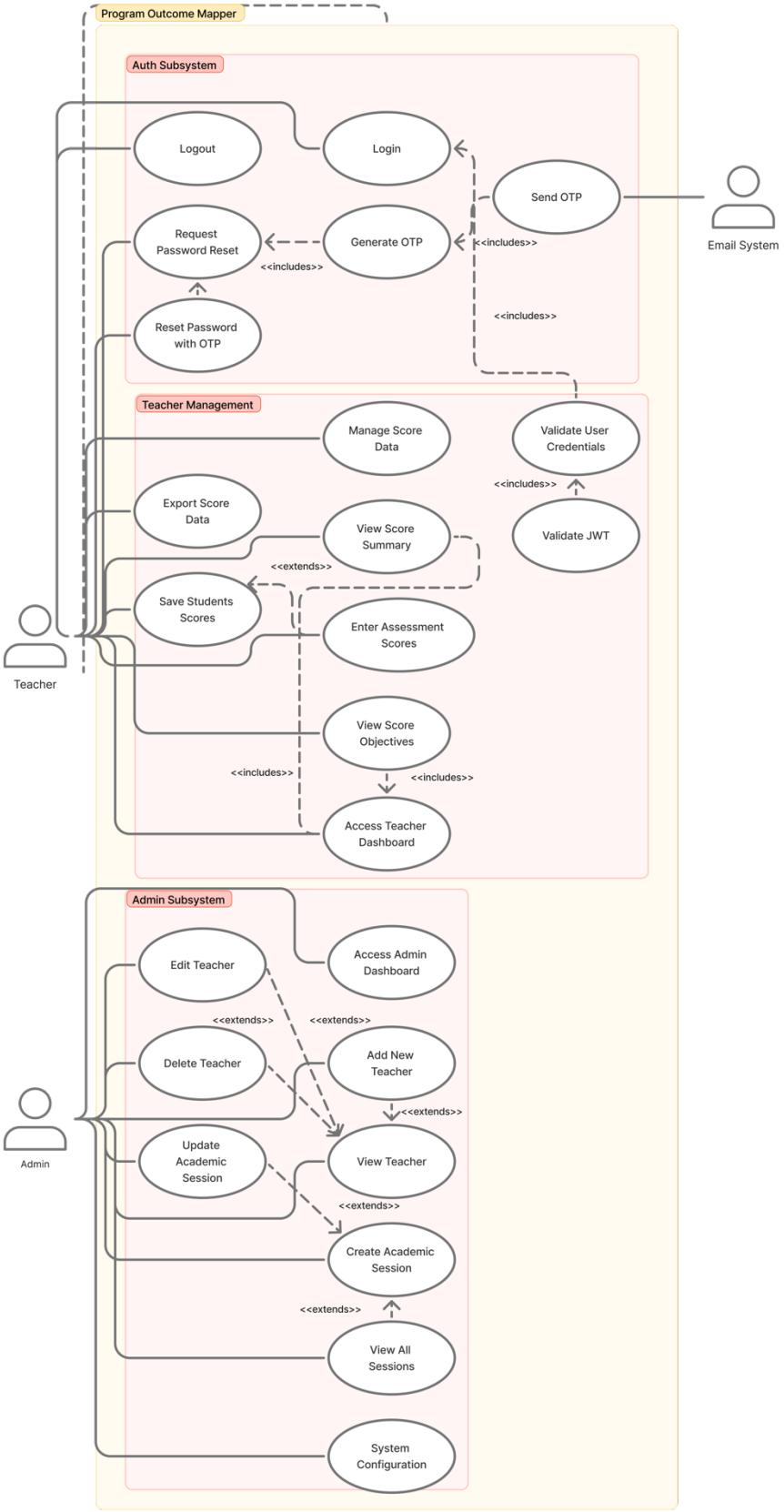
TBD	To Be Determined: Indicates that a decision has not yet been made or information is not yet available.

Appendix B: Analysis Models

Use Case Diagram :

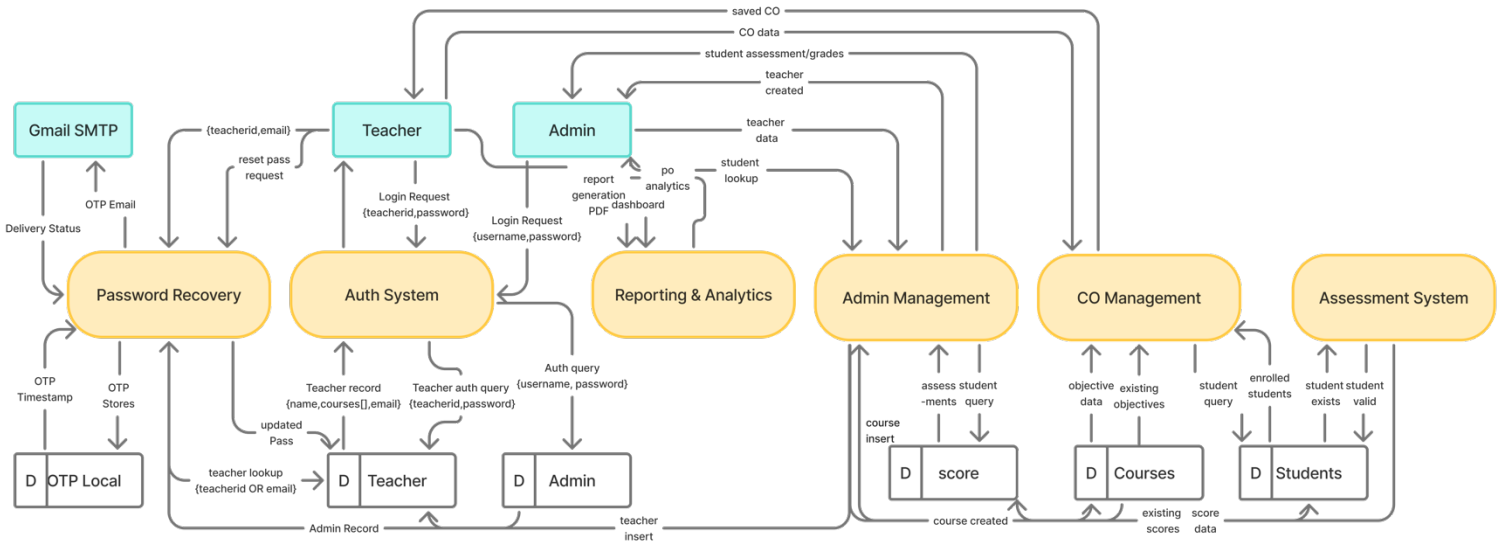
Actors:
Teacher/General User : Primary system users who manage courses and student assessments
Administrator : System administrators with elevated privileges for system management
Email System : External system for sending OTP emails during password reset

Subsystems:
Authentication Subsystem: Handles all user authentication and security functions
Teacher Management Subsystem: Core functionality for teachers to manage academic data
Administration Subsystem: Administrative functions for system management
Core System Functions: Internal system processes supporting all operations


Program Outcome Mapper
Auth Subsystem
Logout
Login
Send OTP
Request Password Reset
Generate OTP
<<includes>>
<<includes>>
Reset Password with OTP
<<includes>>
Email System
Teacher Management
Manage Score Data
Validate User Credentials
<<includes>>
Export Score Data
View Score Summary
Validate JWT
<<extends>>
Save Students Scores
Enter Assessment Scores
Teacher
View Score Objectives
<<includes>>
<<includes>>
Access Teacher Dashboard
Admin Subsystem
Edit Teacher
Access Admin Dashboard
<<extends>>
<<extends>>
Delete Teacher
Add New Teacher
<<extends>>
Admin
Update Academic Session
View Teacher
<<extends>>
Create Academic Session
<<extends>>
View All Sessions
System Configuration

Data Flow Diagram:

This physical DFD represents the actual implementation with specific technologies, database schemas, and API endpoints used in the Program Outcome Mapper System.



External Entities:

- **Teacher:** Frontend users accessing via web browser (React/Next.js)
- **Admin:** Administrative users with elevated privileges
- **Gmail SMTP:** External email service (smtp.gmail.com:587) for OTP delivery

Scrum Methodology for Program Outcome Management System

SDLC Model: Scrum Framework Implementation

Based on the backlog from Jira, our development team implemented **Scrum**, an Agile framework that emphasizes iterative development, collaboration, and continuous improvement. Here's how they organized their development approach:

Epic Structure & Organization
Epic Breakdown

The team organized their product backlog into **4 main Epics**, each representing a major functional area:

Epic 1: User Authentication & Authorization

- **Focus:** Security and access control
- **User Stories:** COUR-9, COUR-10, COUR-11, COUR-12
- **Scope:** Login systems, password recovery, admin access

Epic 2: Admin Dashboard & Management

- **Focus:** Administrative oversight and system management
- **User Stories:** COUR-13, COUR-14, COUR-15, COUR-16
- **Scope:** Program outcome analytics, teacher management, session administration

Epic 3: Teacher & Course Management

- **Focus:** Core academic functionality for educators
- **User Stories:** COUR-17, COUR-18, COUR-19
- **Scope:** Course objectives, student enrollment, academic session management

Epic 4: Student Assessment System

- **Focus:** Assessment and evaluation capabilities
- **User Stories:** COUR-20, COUR-21, COUR-22
- **Scope:** Score entry, summaries, calculations

User Story Writing Format

Standard User Story Structure

As a [User Role]
I want to [Action/Goal]
So that [Business Value/Benefit]

Detailed User Stories Analysis

Authentication Epic Stories

COUR-9

As a Teacher/Admin
I want to log in with my credentials to access the system
So that I can securely access my dashboard and perform my duties

COUR-10

As a Teacher
I want to request a one-time password (OTP) to reset my password

So that I can regain access if I forget my credentials

COUR-11

As a Teacher
I want to reset my password using an OTP
So that I can set a new secure password and continue using the system

COUR-12

As an Administrator
I want to log in through a separate admin portal
So that I can access administrative functions with proper security separation

Admin Dashboard Epic Stories

COUR-13

As an Administrator
I want to view a dashboard with all the program outcome results
So that I can monitor institutional performance and make data-driven decisions

COUR-14

As an Administrator
I want to add new teacher accounts to the system
So that I can onboard faculty members and assign them courses

COUR-15

As an Administrator
I want to view a list of all teachers
So that I can manage faculty information and their course assignments

COUR-16

As an Administrator
I want to create and manage academic sessions
So that I can organize courses by time periods and maintain historical data

Teacher & Course Epic Stories

COUR-17

As a Teacher
I want to view all academic sessions

So that I can select the appropriate time period for my course work

COUR-18

As a Teacher
I want to define and manage objectives for my courses
So that I can map learning outcomes to program objectives effectively

COUR-19

As a Teacher
I want to view the list of students enrolled in my course
So that I can manage assessments and track student progress

Student Assessment Epic Stories

COUR-20

As a Teacher
I want to enter and save student scores for course objectives
So that I can track student achievement against learning outcomes

COUR-21

As a Teacher
I want to view a summary of scores for my students
So that I can analyze class performance and identify areas for improvement

COUR-22

As a System
I want to calculate and store score summaries accurately
So that teachers and administrators have reliable assessment data

Scrum Implementation Strategy

Sprint Organization

- **Sprint Duration:** Likely 2-week sprints based on story complexity
- **Epic Prioritization:** Authentication → Core Features → Advanced Features
- **Story Points:** Estimated based on complexity and effort

Definition of Done

Each user story likely included:

- Feature implementation complete
- Unit tests written and passing
- Integration with database verified
- UI/UX requirements met
- Security requirements satisfied
- Code review completed
- User acceptance criteria verified

Backlog Management

- **Product Owner:** Responsible for prioritizing educational requirements
- **Scrum Master:** Facilitated development process
- **Development Team:** Cross-functional team handling full-stack development

Epic-to-Implementation Mapping

Authentication Epic → Technical Implementation

- **JWT Token System** (using jose library)
- **Cookie-based Session Management**
- **Middleware Route Protection**
- **OTP Email System** (Nodemailer)
- **Password Recovery Workflow**

Admin Epic → Technical Implementation

- **Admin-specific API Endpoints** (/api/admin/*)
- **Role-based Access Control**
- **MongoDB Aggregation Pipelines**
- **Teacher Management CRUD Operations**
- **Session Management System**

Teacher Epic → Technical Implementation

- **Course Objective Management APIs**
- **Student List Management**
- **Session Selection Interface**
- **Dynamic Form Generation**
- **Real-time Validation**

Assessment Epic → Technical Implementation

- **Score Entry System**
- **Assessment Type Management**
- **PDF Report Generation** (jsPDF)
- **Statistical Calculations**

- **Data Aggregation & Analytics**

Benefits of Scrum Approach for Educational Systems

Iterative Development

- Regular feedback from educators
- Incremental feature delivery
- Continuous improvement based on user testing

Stakeholder Collaboration

- Product Owner representing educational needs
- Regular demos to faculty and administrators
- Adaptive planning based on institutional requirements

Risk Mitigation

- Early identification of technical challenges
- Regular integration and testing
- Flexible scope management

Transparency

- Clear progress tracking through story completion
- Regular sprint reviews and retrospectives
- Visible backlog prioritization

This Scrum implementation enabled the team to deliver a comprehensive educational management system while maintaining flexibility to adapt to changing institutional requirements and user feedback throughout the development process.